



Report for the Course Cyber-Physical Systems and IoT Security

Reference paper: Exposing New Vulnerabilities of Error Handling Mechanism in CAN

Andrea Mutti
Stefano Masiero

2026-09-24

1 Objectives

The Controller Area Network (CAN) is the dominant in-vehicle communication bus, yet it was designed without security in mind. The reference paper [Serag et al., 2021] shows that CAN’s error-handling and fault-confinement mechanism, the very feature meant to keep the bus reliable, can be turned into an attack surface. By chaining three newly discovered vulnerabilities the authors build the **Scan-Then-Strike** (STS) attack, whose core offensive primitives are the *Single Frame Bus Off* (SFBO) attack and its escalation into a *Persistent Bus Off* (PBO) denial of service.

This project reproduces those two primitives on a small physical testbed. We pursued three concrete goals:

1. build a real (non-simulated) CAN bus with a controllable victim node, an attacker node, and tunable background traffic, such that bus-off and recovery events are externally observable;
2. implement the SFBO primitive on a microcontroller attacker and verify that a single targeted message can push the victim into the bus-off state;
3. learn the victim’s recovery timing and attempt persistent suppression using two of the paper’s recovery-prevention strategies, open-loop prediction (for fixed recovery) and closed-loop trailing-frame attack (for unpredictable recovery), evaluating how closely a simple ESP32 microcontroller can match the timing precision each requires.

1.1 Project goal and deviations from the paper

This project does not aim to reproduce the full STS attack on a vehicle. Its goal is to implement the timing-critical core of STS and to measure how faithfully a resource-constrained microcontroller can carry it out. While the prototype keeps the protocol-level ideas of the paper, it differs from the original evaluation in three deliberate ways:

- **Hardware platform.** We use ESP32-C3 boards driven by the ESP-IDF TWAI peripheral [Espressif Systems, 2024], whereas the paper used Arduino Uno boards with MCP2515/MCP2551 controllers that give the application tighter low-level control over transmit timing. This single choice turns out to bound what is reproducible (Sec. 5).
- **Attacker knowledge.** Our attacker is given the victim’s target ID a priori instead of discovering it through the network-mapping and victim-identification stages (Stages 1–2), isolating the timing-critical Stages 3–4.
- **Victim recovery.** Our victim enforces a single *fixed* recovery interval for repeatability, rather than the bare-minimum, fixed, sequenced, and random recovery models exhibited by real ECUs in the paper.

These deviations are stated up front because they frame every result that follows: where the hardware allows it the attack reproduces faithfully, and where it does not the gap is attributable to the platform rather than to the method.

2 Background: the CAN protocol

This section summarises the few CAN mechanisms the attack depends on: bitwise arbitration, the error counters, and the three error states.

2.1 Frames and bit encoding

CAN is a multi-master, broadcast bus. A node wishing to transmit sends a *data frame* (Fig. 1) whose first field after the start-of-frame (SOF) bit is an 11-bit **identifier** (ID). The ID does not name a source or destination; it labels the *meaning* of the payload, so a receiver cannot tell who sent a frame, a property the attack later abuses by forging a frame with the victim’s ID. Bits are encoded as **dominant** (*logic 0*) and **recessive** (*logic 1*): if any node drives a dominant bit while another drives a recessive bit, the bus reads dominant. The bus therefore behaves as a *wired-AND*.



FIGURE 1: Standard (11-bit) CAN data frame. Numbers are field lengths in bits. Arbitration is decided over the SOF, identifier and RTR bits.

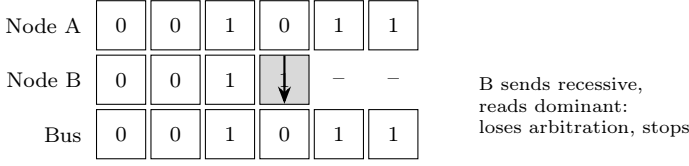


FIGURE 2: Bitwise arbitration. At bit 4 Node B transmits a recessive bit (1) but the bus is dominant (0), so B loses and withdraws while A continues uninterrupted.

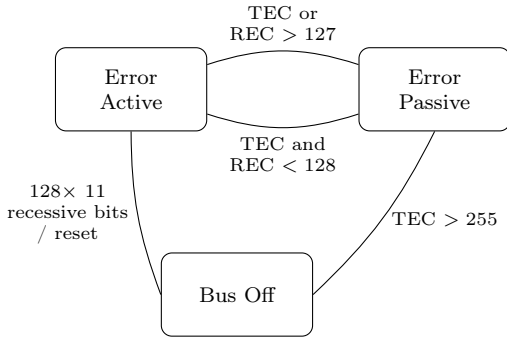


FIGURE 3: CAN error-state machine. The attack drives the victim rightward (active → passive) and then down (passive → bus off) by manufacturing errors faster than successful transmissions can clear them.

2.2 Bitwise arbitration

When two nodes start transmitting at the same time, they arbitrate bit-by-bit over the identifier. Each node monitors the bus as it transmits; a node that sends a recessive bit but reads back a dominant bit has lost arbitration and immediately stops, without corrupting the winning frame (Fig. 2). Because dominant wins, the frame with the numerically smaller ID has higher priority. The same rule extends into the data field, which is why a forged payload with more leading zeros than the victim’s outranks it.

2.3 Error counters and error states

For fault confinement, each node keeps a Transmit Error Counter (TEC) and a Receive Error Counter (REC). A transmission error raises the TEC by 8; a successful transmission reduces it by 1. These counters drive a three-state machine (Fig. 3). A node starts error active and signals errors with a 6-bit dominant *active error frame* that interrupts ongoing traffic. Once a counter exceeds 127 it becomes error passive: it now signals with a *passive error frame* made of 14 recessive bits, which is invisible on an idle-looking bus and does not interrupt others. If TEC exceeds 255 the node goes **bus off**, disconnecting until it observes *128 occurrences of 11 recessive bits* (or is reset).

2.4 Why these mechanisms enable the attack

Two facts above are the levers the paper exploits. First, a deliberate collision (two frames, same ID, different content) forces a bit error and a round of retransmissions, climbing TEC quickly. Second, a passive error frame’s 14 recessive bits are indistinguishable from an idle bus, so any other frame that starts during it interrupts it and regenerates yet another error: **passive error regeneration**. Chaining these lets a single targeted message push a victim all the way to bus off (SFBO), and the recovery behavior lets the attacker re-strike to keep it suppressed (PBO).

3 System Setup

The testbed is a single 500 kbps CAN bus on breadboard wiring, with a twisted CANH/CANL pair and 120 Ω termination at both ends. Three *ESP32-C3 SuperMini* ECUs share the bus (Fig. 4), each with its own *SN65HVD230* CAN transceiver, and a Linux PC connected through a USB-to-CAN (uCAN) adapter exposed as a SocketCAN interface. Each module is described in its own subsection below.

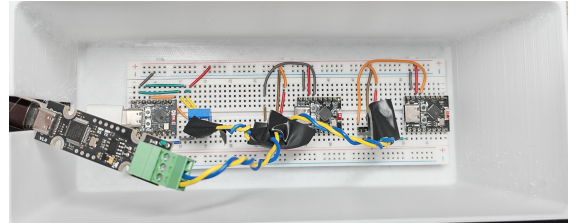


FIGURE 4: Physical testbed: attacker, victim and background-traffic ESP32-C3 nodes on a terminated 500 kbps bus, orchestrated by a Linux host over a uCAN adapter.

3.1 Attacker node

The attacker is an ESP32-C3 running `attacker.ino`. This is the only node that performs the offensive primitives: SFBO, the persistent re-attack, and the recovery-learning logic. Internally it is a state machine:

IDLE → MEASURE → LEARN → PERSIST → DONE

It starts by a trigger frame (0x600) sent from the PC. So that its progress is observable externally, after each phase it transmits an 8-byte report frame on ID 0x601 encoding the total attempts, the number of learned bus-offs, the current state, the persistence bus-off count, and the run index; the PC logger decodes this to mirror the attacker’s view. The attacker fully reinitialises its CAN controller after every attack, a reliability workaround whose timing cost is analysed in Sec. 4.5.

3.2 Victim node

The victim is an ESP32-C3 running `victim.ino`. It transmits two standard IDs, 0x7A and 0x7F, alternating every 5 ms with a 3-byte payload (11 22 33 and 44 55 66 respectively), so each ID appears once every 10 ms; the attacker locks onto the 0x7A signature (dlc=3, data 11 22 33). The firmware uses two FreeRTOS tasks (FreeRTOS is the real-time multitasking kernel the ESP-IDF runs on, which schedules concurrent tasks by priority [FreeRTOS, 2024]): a high-priority alert task blocking on `twai_read_alerts()` that reacts to bus-off and recovery events within tens of microseconds, and a lower-priority task driving the periodic transmissions. A heartbeat GPIO and serial logging expose the controller state. The victim’s recovery policy is made deterministic on purpose; the mechanism is detailed in Sec. 4.4.

3.3 Background-traffic node

The third ESP32-C3 runs `generator.ino` and emits high-rate traffic on IDs 0x110/0x111. Its purpose is to raise the bus load and create realistic arbitration jitter. Crucially, both IDs are numerically larger than the victim’s 0x7A, i.e. *lower* priority, so this node does not defer the victim’s transmissions. This contrasts with the PC-side load described next, and the distinction becomes important in Sec. 4.7.3.

3.4 Linux controller and logger

The Linux PC runs a custom C program, `pc_logger.c`, over SocketCAN [The Linux Kernel, 2024]. It performs three jobs:

1. it injects additional background load on IDs 0x54–0x57 at a configurable interval (default 200 μ s, contributing $\approx 85\%$ of the bus load);
2. it sends the 0x600 trigger that starts the attack;
3. it traces every frame, detects victim “silence” windows (>50 ms gaps), measures recovery intervals from the trace and decodes the 0x601 report frames.

It is just an orchestrator and logger, not the attacker. This choice is forced due to the raw SocketCAN, that cannot reliably perform the bit-level collision timing SFBO requires, which is why the attack itself runs on the ESP32. Note that the default load IDs (0x54 - 0x57) are higher priority than the victim’s 0x7A; this choice is revisited in Sec. 4.7.3, where load priority turns out to matter.

3.5 Threat model and assumptions

We adopt the paper’s threat model in a reduced form. The attacker is a compromised node already on the bus, able to run arbitrary code on its own CAN controller, but with no privileged access to the victim and no ability to read or write the victim’s internal state. As stated in the Objectives, the attacker is given the victim’s target ID a priori instead of mapping the network; this is the only assumption we relax. The attacker respects the constraints of commodity CAN hardware: it cannot drive individual bits onto the wire and cannot guarantee the exact transmit instant of a frame, which is the central practical limitation explored in Sec. 5.

3.6 CAN bus parameters

The bus runs at a nominal bit rate of 500 kbps where one bit period is 2 μ s. A standard (11-bit) data frame carrying d data bytes occupies $44 + 8d$ fixed-form bits, plus the 3-bit inter-frame space (IFS) and any stuff bits. This gives the frame durations in Table 1. The victim frame ($d = 3$) is $\approx 142 \mu$ s; a preceded frame is $\approx 244 \mu$ s; and a zero-length frame is $\approx 94 \mu$ s, matching the figure used in the paper [Serag et al., 2021].

Frame	Bits (+IFS)	Duration
Zero-byte ($d=0$)	47	$\approx 94 \mu$ s
Victim ($d=3$)	71	$\approx 142 \mu$ s
8-byte ($d=8$, nom.)	111	$\approx 222 \mu$ s
Preceding ID ($d=8$)	122	$\approx 244 \mu$ s
8-byte (worst stuffing)	~ 135	$\approx 270 \mu$ s

TABLE 1: Frame durations at 500 kbps used to calibrate the attack timing. Stuff bits add up to ~ 24 bits in the worst case for an 8-byte frame.

4 Experiments

This section describes the methodology used to implement and evaluate the adapted Scan-Then-Strike attack.

4.1 Methodology

The experiment follows the timing-then-interference logic of STS, reduced to a known victim ID:

1. **Measure:** the attacker passively times the victim’s 0x7A frame over 256 samples and records its period and payload signature.
2. **Learn:** it launches repeated SFBO instances, and for each successful bus-off it measures the recovery interval, until 30 successful bus-offs are collected. The median is taken as the recovery estimate.
3. **Persist:** it attempts to keep the victim suppressed with one of two strategies (Sec. 4.7.4).

The Linux logger runs throughout, providing an independent view of victim presence/absence to cross-check the attacker’s self-reported state.

4.2 SFBO primitive and its mapping to the paper

A single SFBO attempt follows these steps:

1. waits for a victim frame to anchor timing;
2. sleeps until `ATTACK_ADVANCE_US` (235 μ s) before the next expected slot;
3. emits a single high-priority preceded-ID frame (0x001);
4. transmits a same-ID, higher-priority forgery (0x07A with payload 00 00 00) to force the collision;
5. follows with 15 back-to-back clutter frames (0x777) to exploit passive-error regeneration and drive the victim’s TEC to 256.

Each element maps onto a specific construction in the paper. The single preceded ID is the paper’s Appendix A synchronisation primitive: a higher-priority frame transmitted immediately before the victim’s slot forces both nodes to begin transmitting at its conclusion, producing the collision. The forged payload 00 00 00 has more leading zeros than the victim’s 11 22 33, so it is “higher-priority content” and wins the data-field arbitration, forcing the victim into a bit error. The clutter ID is arbitrary, exactly as the paper specifies. The choice of 15 clutter frames, instead, is not arbitrary: the paper shows that after Step-3 the victim sits near $TEC = 136$ and each clutter frame regenerates a passive error worth +8, so $TEC = 136 + 8 N_{\text{clutter}}$; with $N_{\text{clutter}} = 15$ this reaches 256, leading the victim TEC to 256, forcing it into bus-off. Our $N_{\text{CLUTTER}} = 15$ is therefore the paper’s minimal value.

4.3 Calibrating ATTACK_ADVANCE_US and the preceded ID

The hardest part of SFBO on this platform is hitting the victim’s transmit slot. `ATTACK_ADVANCE_US` sets how early the attacker queues its preceded frame. It must be roughly one frame-time, so that the preceded frame ($\approx 244 \mu\text{s}$) occupies the bus just before the victim’s expected arrival t_{nom} (Table 1); the victim then finds the bus busy with the higher-priority preceded frame and releases its own frame at the preceded frame’s conclusion, synchronised with the attacker’s forgery. We use $235 \mu\text{s} \approx$ one 8-byte frame time ($244 \mu\text{s}$ with 11 stuff bits) with a small margin. The residual was tuned empirically to absorb the TWAI TX-queue latency, the unpredictable delay between calling `writeFrame()` and the frame appearing on the wire, which the bit-timing calculation cannot capture. During learning a single preceding frame ($N_{\text{PRECEDED_LEARN}} = 1$) is sufficient, because each attack is anchored on a freshly observed victim frame and the timing uncertainty is small.

The blind prediction-mode persistence (Sec. 4.7.4) is different: it fires without a fresh observation, so its predicted instant carries much larger uncertainty and the victim has a greater uncertainty on the recovery time.

To compensate, that mode uses a wide train of $N_{\text{PRECEDED_BLIND}} = 11$ preceded frames ($\approx 1 \text{ ms}$), blanket-ing the window around the predicted recovery rather than relying on a single precisely placed frame. The blind attack is also launched `PERSIST_LEAD_US` = $725 \mu\text{s}$ before the predicted recovery instant, so the preceded train brackets the expected recovery rather than trailing it.

4.4 Victim implementation

A real ECU recovers from bus-off after a vendor-specific interval that may be bare-minimum, fixed, sequenced, or random [Serag et al., 2021]. To make experiments repeatable we constrained the victim to a *fixed* model: on a `TWAI_ALERT_BUS_OFF` event the high-priority alert task captures an anchor with `esp_timer_get_time()`, initiates recovery, and then holds the first post-recovery transmission until a fixed 100 ms have elapsed from the anchor. The 100 ms value is an arbitrary, round choice for repeatability; the only requirement was a constant, easily learnable interval. Because recovery is timer-driven rather than tied to counting recessive bits, our victim’s recovery interval is independent of bus load,

unlike the bare-minimum nodes in the paper, whose recovery slows as the bus fills.

4.5 Reliability workaround

After every attack the attacker fully reinitialises its CAN controller (`ESP32Can.end()`, a short delay, then `begin()`). Without this, the TWAI peripheral repeatedly got stuck after heavy error bursts and stopped transmitting. The reinit reliably clears that state, but it costs $\approx 25\text{--}30 \text{ ms}$ of variable dead time per attack, a cost that turns out to matter for prediction-mode persistence (Sec. 5). We found empirically that after a heavy error burst the TWAI peripheral did not resume normal transmission on its own: it would not reliably clear its accumulated error state by sending successful frames, and instead stalled until fully reinitialised.

4.6 Data collection

Two independent sources were used: the attacker’s serial log (period, per-phase recovery times, persistence progress) plus logger 0x601 report decoding. Having both lets us attribute any discrepancy either to the attacker’s internal view or to what was actually observable on the wire.

4.7 Tests

We evaluated the prototype through four tests that follow the attack pipeline: period measurement, single-frame bus-off (learning), bus-load sensitivity, and persistence. Each is described below with the representative output it produced.

4.7.1 Period measurement and victim signature

The attacker first times the victim and locks onto its payload signature. Across repeated 256-sample measurements the estimate was 10.00 ms (individual readings 10000/9999/10000 μs), with the correct 3-byte signature 11 22 33. This high stability is expected: at $\approx 79\%$ load the victim’s 0x7A frame is mid-priority and only lightly deferred, so its period is recovered almost exactly. This load is achieved with the logger sending frames at 500 μs distance each.

4.7.2 Learning: single-frame bus-off reliability

The learning stage launches SFBO repeatedly and records the recovery interval after each success, until 30 bus-offs are collected (Listing 1). It reached the target, with the recovery distribution summarised in Table 2. The measured median, $\approx 101 \text{ ms}$, equals the victim’s 100 ms hold plus $\approx 1 \text{ ms}$ of detection overhead, confirming that the deterministic victim behaves as intended and that the recovery model is cleanly learnable, the premise of Stage 3.

LISTING 1: Trimmed attacker serial log of the learning stage. Recovery times are reported per successful bus-off; the run completes with a learned median.

```
[TASK] triggered
[MEASURE] collecting 256 samples for
          0x07A...
[MEASURE] victim signature dlc=3
          data=11 22 33
[MEASURE] avg=10000 us good=256
[TASK] period=10000 us — LEARNING
```

```

    (need 30 bus-offs)
[LEARN] phase 1/30, attempt 1/2
[SFBO] *** BUS-OFF *** recovery=100972 us
[LEARN] phase 2/30, attempt 1/2
[SFBO] *** BUS-OFF *** recovery=101097 us
... (phases 3–15 succeed) ...
[LEARN] phase 16/30, attempt 1/2
[SFBO] miss
[LEARN] phase 16/30, attempt 2/2
[SFBO] miss
[LEARN] exceeded 2 attempts, resetting
        phase ...
... (re-measure, then phases 16–30
    succeed) ...
[LEARN] phase 30/30, attempt 1/2
[SFBO] *** BUS-OFF *** recovery=100979 us
[TASK] LEARN DONE: median
        recovery=100972 us

```

Quantity	Value
Victim period (measured)	10.00 ms
Target bus-offs	30
Bus-offs achieved	30
SFBO attempts (incl. misses)	≈36
Single-shot success rate	≈83%
Recovery min	100.12 ms
Recovery median	100.97 ms
Recovery max	101.64 ms

TABLE 2: Learning-stage results. SFBO reliably forces single bus-offs; the fixed-recovery victim yields a ≈1.5 ms-wide recovery distribution.

Whenever a learning phase misses 2 consecutive attempts, the controller is restarted, and another measurement is performed to realign with the victim, to ensure reliability even after possible changes on the victim side.

4.7.3 Bus-load sensitivity

Beyond the baseline we swept the bus load to test the paper’s claim that external traffic helps the attack, varying two factors independently: the *amount* of background traffic (logger interval and the 0x110/0x111 generator) and its *priority relative to the victim* (load IDs below or above 0x7A). Table 3 reports the trend.

Load amount	Dominant priority	SFBO reliability
Low	—	low
Medium	below victim	high
High	below victim	highest
High	above victim	degraded

TABLE 3: Qualitative effect of bus load on SFBO reliability. Load amount helps.

Amount helps. Raising the total load increased SFBO reliability to reach bus-off. This is exactly the paper’s observation that external traffic “will not interrupt but instead help” the attack [Serag et al., 2021]: any frame that begins transmitting during the victim’s regenerated passive error frame

(14 recessive bits, indistinguishable from idle) interrupts it, causing a form error and another +8 on the victim’s TEC. External traffic therefore acts as free clutter, accelerating the climb to 256.

Priority could increase jitter. The refinement is that the *priority* of the added load matters. When the load was dominated by IDs higher-priority than the victim, SFBO reliability dropped. The cause is arbitration deferral: a victim frame that finds higher-priority traffic pending must wait, so its actual on-bus time drifts away from the nominal slot the attacker predicted from the period. This is the same effect the paper reports in Sec. 6.2, where low-priority messages exhibited higher jitter ($\sigma \approx 0.6$ ms) because higher-priority traffic deferred them. The most favorable condition is therefore a mixture of high load of lower priority and a low load of higher priority messages, which is exactly what we achieved here, after having done various tests with different configurations.

4.7.4 Persistence: two recovery-prevention strategies

The attacker implements two recovery-prevention strategies from the paper, selected by the `USE_FIXED_ATTACK` flag.

Prediction mode (open-loop). With `USE_FIXED_ATTACK = true` the attacker predicts the next recovery instant from the learned median and fires a blind SFBO (the 11-frame preceded train of Sec. 4.3) a small lead (`PERSIST_LEAD_US`) before it, without re-observing the victim. This is the paper’s strategy for the fixed recovery models (Sec. 5.4). When the persistence lands the victim is fully silenced, a true persistent bus-off with no frames in between, but the cost to pay is that the loop is open: per-cycle timing error accumulates and the chain breaks after a few hops. Across five runs (Table 4) the target of 20 consecutive bus-offs was almost never reached; on average, every run reaches from 9–10 consecutive persistence bus off. (Listing 2).

LISTING 2: Prediction mode: the blind chain advances a few hops, then a miss forces a restart; the run fails after the restart budget is spent.

```

[PERSIST] starting 30 runs (target 20, median=100972 us, mode=FIXED)
[PERSIST] ===== Run 5/30 =====
[PERSIST] initial bus-off succeeded (count=1)
[PERSIST] bus-off #2
[PERSIST] bus-off #3
[PERSIST] bus-off #4
[PERSIST] bus-off #5
[PERSIST] bus-off #6
[PERSIST] bus-off #7
[PERSIST] bus-off #8
[PERSIST] bus-off #9
[PERSIST] bus-off #10
[PERSIST] miss at count 10, restarting...
... (further restarts, none reach 20) ...
[PERSIST] run 5 failed (max restarts reached)

```

Trailing mode (closed-loop). With `USE_FIXED_ATTACK = false` the attacker does not use the prediction strategy. It waits to observe the victim’s recovery frame, then attacks the *trailing* frame one period later, re-anchoring its timing on a fresh observation every cycle. This is the paper’s strategy for the *random* recovery model (Sec. 5.3 and Appendix B.2): the first recovery message is deliberately allowed through and used only as a recovery signal, while the next (trailing) message of the same ID is collided. Listing 3 shows the

Run	Longest chain	Reached 20?
1	8	no
2	12	no
3	9	no
4	20	yes
5	10	no

TABLE 4: Prediction-mode persistence runs, with 5 runs representative of 30. The open-loop re-attack drifts out of the collision window after a few hops.

resulting bus trace: the victim’s genuine 11 22 33 frame leaks once per cycle and the attacker’s forced 00 00 00 frame follows ≈ 15 ms later, pushing the victim back to bus-off.

LISTING 3: Trailing mode bus trace (candump). The victim’s genuine frame (11 22 33) reappears once per recovery cycle and is immediately followed by the attacker’s forced collision frame (00 00 00).

```
(18:23:10.389217) can0 07A [3] 00 00 00 <- attacker (forced)
(18:23:10.488794) can0 07A [3] 11 22 33 <- victim recovery frame (
leaks)
(18:23:10.503793) can0 07A [3] 00 00 00 <- attacker hits trailing
(18:23:10.602862) can0 07A [3] 11 22 33 <- victim recovery frame
(18:23:10.617585) can0 07A [3] 00 00 00 <- attacker hits trailing
```

From the trace, the victim’s genuine 0x7A frame reappears every ≈ 114 ms (two successive 11 22 33 frames at +10.488794 and +10.602862 s), against a nominal 10 ms period. The target ID’s genuine throughput is therefore reduced by $1 - 10/114 \approx 91\%$. Unlike prediction mode this does not produce complete silence, because one frame leaks per cycle by design, but it is sustained: because each attack re-anchors on an observed frame there is no accumulating drift, so the suppression continues indefinitely rather than collapsing after a few hops.

Comparison. Table 5 summarises the trade-off. Both modes fire the identical SFBO primitive once per cycle; what differs is what each cycle anchors on. Prediction mode is open-loop: it computes each shot from the learned median without re-observing the victim, so per-cycle timing error accumulates and the chain breaks after a few hops. Trailing mode is closed-loop: it waits for the leaked recovery frame and re-anchors on it every cycle, so each shot is as well-synchronised as a learning-phase SFBO ($\approx 83\%$ success) and nothing accumulates.

Mode	Prediction	Trailing
Re-attack		
basis	predicted time	observed frame
Loop	open	closed
Victim fully		
silenced	yes (per hit)	no
Sustained	no (9–10 hops)	yes
Target		
suppression	100% briefly	$\approx 91\%$
Paper model	fixed	random

TABLE 5: The two persistence strategies and their trade-offs on the ESP32 testbed, with instantaneous best-cases

4.8 Code availability

The complete firmware and logger source are provided with the submission. The exact version used for these experiments is published at this fresh repository:

<https://github.com/stefano-masiero/Scan-Then-Strike>

5 Results and Discussion

The tests show a clear split: the single-shot primitive and the recovery learning reproduce faithfully, the open-loop persistent attack does not, and the closed-loop trailing attack does, at the cost of partial silence. This section explains why, and positions each outcome against the paper.

5.1 Why prediction-mode persistence is not reliable

SFBO is a single, retryable event aimed at a window we anchor on a freshly observed victim frame, so an $\approx 83\%$ per-shot success rate is more than enough to learn the recovery model. Prediction-mode persistence is fundamentally harder because it is **open-loop**: after the first bus-off the attacker stops observing the victim and fires each subsequent attack at an instant computed from the learned median. Any per-cycle timing error accumulates, and after a few hops the predicted instant no longer overlaps the victim’s recovery frame, so the chain breaks, as the data show.

Two platform factors dominate the per-cycle error. First, the ESP32 TWAI peripheral exposes only a high-level frame interface: as the paper itself notes, application code “cannot precisely control the transmission time of a message” [Serag et al., 2021]. FreeRTOS scheduling and the driver’s TX queue add jitter on top. Second, the controller-reinit workaround that makes SFBO more robust is what makes prediction-mode persistence fail: each ≈ 25 –30 ms reinit injects a large, variable delay between the timing anchor and the next blind shot, and over up to 20 cycles this variance swamps the collision window. The 11-frame preceded train widens that window but cannot fully absorb the accumulated drift.

5.2 Why trailing-mode persistence achieves better results

Trailing mode removes the open-loop drift entirely. By waiting for the victim’s recovery frame and re-anchoring on it every cycle, it converts the attack back into a sequence of observed SFBO instances, the same well-synchronised event that succeeds $\approx 83\%$ of the time in learning. There is therefore nothing to accumulate, and the suppression is sustained. The price is structural, not a defect: the strategy *must* let the first recovery frame through in order to detect recovery, so the victim leaks one genuine frame per cycle. This is the behaviour the paper describes for the random model, where “the victim will be successful at transmitting the first recovery message at every recovery attempt” [Serag et al., 2021]. Our trailing-mode trace is the direct empirical realisation of that statement.

Notably, this confirms in practice the fix we anticipated for the open-loop failure: a closed-loop, re-observing re-attack.

5.3 Coherence with the reference paper

Our results line up with the paper wherever the hardware allows it, and the deviations are well understood:

- **SFBO single-frame bus-off:** reproduced. A single targeted message, synchronised with one preceded ID exactly as in Appendix A, reliably forces bus-off, and the clutter count follows the paper’s $TEC = 136 + 8 N_{\text{clutter}}$ rule.
- **Deterministic recovery:** reproduced for the fixed model. The recovery is learnable to ≈ 1.5 ms.
- **Load helps the attack:** reproduced, with the added nuance that load priority modulates the effect through victim jitter, itself a paper observation (Sec. 6.2).
- **Fixed recovery prevention (prediction):** partially reproduced. The mechanism works per hop but is not sustained, because the ESP32/TWAI stack cannot hold microsecond synchronization across many open-loop cycles. We still achieved a suppression rate $\approx 70\%$ using the fixed recovery prevention method.
- **Random recovery prevention (trailing):** reproduced. Sustained $\approx 78\%$ suppression of the target ID, with the expected single leaked recovery frame per cycle, comparable to the paper’s vehicle random-recovery rates (TCM 85%, ECM 83%), and plausibly slightly lower here because our bus-off reliability is slightly lower, as discussed above.

5.4 Limitations and outlook

The prototype remains a reduced testbed: it skips network mapping and victim identification, targets a single known ID, and uses a victim with an artificially clean fixed recovery. The one genuine reproduction gap is mostly a hardware timing limit; closing it would require tighter low-level control than the TWAI peripheral exposes (e.g. an MCP2515-class controller as in the paper) or avoiding the full controller reinit so that per-cycle latency becomes predictable. The testbed cleanly separates victim timing policy from attacker logic, so either change can be tested without touching the measurement pipeline.

6 Conclusions

We built a CAN testbed composed of three ECUs and one logger and implemented the timing-critical core of the STS attack on ESP32-C3 hardware. The SFBO primitive reliably forces single bus-offs ($\approx 83\%$ per shot), using the paper’s single preceded-ID synchronisation and its $TEC = 136 + 8 N_{\text{clutter}}$ clutter model. Against a deterministic fixed-recovery victim the recovery model is learned with ≈ 1.5 ms precision (median 100.97 ms over 30 bus-offs), and the bus-load sweep confirms that traffic aids the attack while revealing that load priority sets the limit through victim jitter. For persistence we implemented two of the paper’s recovery-prevention strategies. Open-loop prediction produces true silence per hit but cannot be sustained on this hardware, because the ESP32/TWAI stack and the necessary controller-reinit workaround prevent microsecond synchronisation across open-loop cycles. Closed-loop trailing attack, by re-anchoring on each observed recovery

frame, sustains $\approx 78\%$ suppression of the target ID with one leaked frame per cycle, almost the exact behaviour the paper predicts for the random recovery model, and an empirical confirmation that closed-loop re-anchoring is the way around the open-loop timing limit.

References

- [Espressif Systems, 2024] Espressif Systems (2024). Two-wire automotive interface (twai) — esp-idf programming guide. Accessed 2026-06-09.
- [FreeRTOS, 2024] FreeRTOS (2024). Task management — freertos kernel.
- [Serag et al., 2021] Serag, K., Bhatia, R., Kumar, V., Celik, Z. B., & Xu, D. (2021). Exposing new vulnerabilities of error handling mechanism in can. In 30th USENIX Security Symposium (USENIX Security 21).
- [The Linux Kernel, 2024] The Linux Kernel (2024). Socket-can — controller area network.